SOLID Principles

Clean Object-Oriented Design, explained simply — with Java

S	O	L	I	D
Single Responsibility	Open / Closed	Liskov Substitution	Interface Segregation	Dependency Inversion

SOLID is five design principles that share one goal: **reduce how far a single change ripples through your code**. Get them right and one edit stops breaking ten other things. Each principle below has a plain definition, a real-world analogy, and a bad → good Java example you can run.

What's inside

S Single Responsibility Principle

SRP — one class, one job

Definition

A class should have only **one reason to change**. In plain words: one class, one job.

The key is the phrase **“reason to change”**. If two different people, for two unrelated reasons, would both come asking you to edit a class, that class is doing too much.

Real-world analogy

One employee who is the chef, the accountant, *and* the cleaner. Tax rules change — you bother them. The menu changes — you bother them. Cleaning supplies change — you bother them again. Three unrelated reasons land on one person, and if they quit, everything collapses at once. Better: hire a chef, an accountant, and a cleaner, each changing independently.

One class doing three unrelated jobs — it changes for the tax rules, the database, *and* the print layout:

✗ BAD

```
class Invoice {
    void calculateTotal() {
        // business logic: add line items, apply tax
    }
    void saveToDatabase() {
        // database logic: open connection, write rows
    }
    void printInvoice() {
        // formatting logic: layout, fonts, spacing
    }
}
// 3 unrelated reasons to change = 3 responsibilities
```

Split by reason to change. Now a database switch touches only the repository; the tax math sits untouched and safe:

✓ GOOD

```
class Invoice {
    void calculateTotal() { /* ONLY business logic */ }
}

class InvoiceRepository {
    void save(Invoice invoice) { /* ONLY database logic */ }
}

class InvoicePrinter {
    void print(Invoice invoice) { /* ONLY formatting */ }
}
```

The smell to remember

A class that changes for many **unrelated reasons**. Note: “one responsibility” doesn't mean “one method” — many methods are fine as long as they all serve the *same* job.

O Open / Closed Principle

OCP — extend, don't edit

Definition

Software should be **open for extension but closed for modification**. Add new behaviour by **writing new code**, not by editing old, tested code.

Old code is tested code. Every time you reopen a working class to bolt on a new case, you risk breaking what already worked.

Real-world analogy

A **power strip**. To add a new lamp you plug it into a free socket — you don't unscrew the wall and rewire the house. The strip was designed to accept new devices without being modified.

Every new payment type forces you to reopen and edit `pay()`. Each edit risks breaking the card path that already worked:

✗ BAD

```
class PaymentProcessor {
    void pay(String type, double amount) {
        if (type.equals("CreditCard")) {
            // credit card logic
        } else if (type.equals("UPI")) {
            // UPI logic
        }
        // PayPal? Wallet? EDIT this method AGAIN.
    }
}
```

Define a contract, let each type be its own class. `PaymentProcessor` never changes again — a new feature is a new class:

✓ GOOD

```
interface PaymentMethod {
    void pay(double amount);
}

class CreditCardPayment implements PaymentMethod {
    public void pay(double amount) { /* card logic */ }
}

class UpiPayment implements PaymentMethod {
    public void pay(double amount) { /* UPI logic */ }
}

class PaymentProcessor {
    void process(PaymentMethod method, double amount) {
        method.pay(amount);   // doesn't care WHICH method
    }
}

// Add PayPal = ONE new class, nothing else edited:
class PayPalPayment implements PaymentMethod {
    public void pay(double amount) { /* PayPal logic */ }
}
```

The smell to remember

Adding a feature means editing an old `if / else` chain. Apply OCP where you already see variation coming (payment types, shapes, report formats) — don't abstract everything “just in case”.

L Liskov Substitution Principle

LSP — subclass keeps the promise

Definition

If code expects a **parent** type, you should be able to hand it **any subclass** and everything still works. A child must honour the **promises** its parent makes.

Inheritance isn't just “looks similar”. A subclass that breaks what the parent guarantees is a fake relationship, even if it compiles.

Real-world analogy

I hand you a box labelled **“everything here flies”**. You trust the label and throw any item expecting flight. If I sneak a **rock** into that box, it drops — the problem isn't the rock, it's that I put it in a box that *promised* flight. A penguin in a `Bird.fly()` world is that rock.

A `Penguin` is-a `Bird` on paper, but substituting it breaks code that worked for every other Bird:

✗ BAD

```
class Bird {
    void fly() { System.out.println("Flying"); }
}

class Penguin extends Bird {
    void fly() {
        throw new UnsupportedOperationException(
            "Penguins can't fly!");   // broken promise
    }
}

void makeItFly(Bird bird) { bird.fly(); }
makeItFly(new Sparrow());   // works
makeItFly(new Penguin());   // CRASHES at runtime
```

Pull the flying promise out of `Bird`. Only real flyers wear `Flyable`. The bad substitution moves from a runtime crash to a compile-time block — impossible to write by accident:

✓ GOOD

```
class Bird {
    void eat() { System.out.println("Eating"); }
}

interface Flyable {
    void fly();
}

class Sparrow extends Bird implements Flyable {
    public void fly() { System.out.println("Flying"); }
}

class Penguin extends Bird {        // never claims to fly
    void swim() { System.out.println("Swimming"); }
}

void makeItFly(Flyable creature) { creature.fly(); }
makeItFly(new Sparrow());    // OK
// makeItFly(new Penguin()); // won't even COMPILE
```

The smell to remember

A subclass that overrides a method just to **throw an exception** or **leave it empty** (“this doesn't apply to me”). That's the loudest LSP warning sign.

I Interface Segregation Principle

ISP — small interfaces, not fat ones

Definition

Don't force a class to implement methods it doesn't need. Prefer **many small, focused interfaces** over **one big fat one**.

An interface is a contract — a to-do list a class must fully complete. If the list has items that make no sense for some classes, the list is too big. Split it.

Real-world analogy

A restaurant with **one giant fixed combo** — starter + main + dessert + drink, all forced together. Someone who only wants a drink is stuck “ordering” food they'll throw away. Better: separate menus, take only what applies.

One fat interface forces the robot to implement `eat()` — something it can't meaningfully do, so it throws or fakes it:

✗ BAD

```
interface Worker {
    void work();
    void eat();
}

class Robot implements Worker {
    public void work() { /* robot working */ }
    public void eat() {
        // robots don't eat... but FORCED to implement
        throw new UnsupportedOperationException();
    }
}
```

Break the big contract into small ones. Each class signs only the contracts that apply to it:

✓ GOOD

```
interface Workable {
    void work();
}

interface Eatable {
    void eat();
}

class Human implements Workable, Eatable {
    public void work() { /* working */ }
    public void eat()  { /* eating  */ }
}

class Robot implements Workable {   // only what it needs
    public void work() { /* working */ }
}
```

The smell to remember

A class implements an interface but some methods end up **empty** or **throwing “not supported”**.
(Same alarm bell as LSP — but here it's about contract size, not substitution.)

D Dependency Inversion Principle

DIP — depend on interfaces, inject from outside

Definition

High-level code should depend on **abstractions (interfaces)**, not on **concrete classes**. And the specific thing should be **plugged in from outside**, not created inside.

Your important business logic shouldn't be hard-wired to one exact tool. It should ask for a *type* of tool (an interface), and you hand it the real one when you run it.

Real-world analogy

A **lamp with a plug**. It fits a standard socket — any compliant power source works, anywhere. If the lamp were soldered straight to one power line, you could never move or change it. That soldering is hard-wiring a concrete class.

Two problems: the service depends on a concrete class, and it news that class itself — soldered to email forever:

✗ BAD

```
class EmailSender {
    void send(String msg) {
        System.out.println("Email: " + msg);
    }
}

class NotificationService {
    private EmailSender sender = new EmailSender(); // soldered
    void notifyUser(String msg) { sender.send(msg); }
}
// Locked to email. Want SMS? Reopen and edit this class.
```

Depend on an interface, hand the real sender in from outside (“injection”). `NotificationService` now works with any sender you plug in:

✓ GOOD

```
interface MessageSender {
    void send(String msg);
}

class EmailSender implements MessageSender {
    public void send(String m){ System.out.println("Email: "+m);}
}
class SmsSender implements MessageSender {
    public void send(String m){ System.out.println("SMS: "+m);}
}

class NotificationService {
    private final MessageSender sender;
    NotificationService(MessageSender sender) { // injected
        this.sender = sender;
    }
    void notifyUser(String msg) { sender.send(msg); }
}

new NotificationService(new EmailSender()).notifyUser("Hi");
new NotificationService(new SmsSender()).notifyUser("Hi");
```

The smell to remember

Seeing `new ConcreteClass()` **inside** an important business class. It's soldered — pull it into an interface and inject the real one from outside.

All five at a glance

	Principle	Memory hook	The smell it fixes
S	**Single Responsibility**	one class, one job	a class changed for many unrelated reasons
O	**Open / Closed**	extend, don't edit	adding a feature means editing old if/else
L	**Liskov Substitution**	subclass keeps the promise	overriding a method to throw / empty it
I	**Interface Segregation**	small interfaces, not fat	empty / throwing methods from a big interface
D	**Dependency Inversion**	depend on interfaces	new ConcreteClass() inside business logic

The one thread

*Every principle is a different tactic for the **same goal**: reduce how far a single change ripples through your code, so one edit can't break ten things. Notice how the fixes repeat — splitting a capability into its own interface shows up in L, I, and D. The principles reinforce each other.*